

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

DIVISIÓN DE CIENCIAS DE LA INGENIERÍA

ÁREA PROFESIONAL

Software Avanzado

Docente: Ing. Moisés Granados



Práctica 5: Saga Pattern vs Circuit Breaker Pattern

Carné

201430801

Nombre:

Luis Basilio Baquixax Sic

Contenido

1. Introducción.....	3
2. Objetivos.....	4
3. Marco teórico.....	5
3.1 Patrón Saga.....	5
3.2 Patrón Circuit Breaker.....	5
4. Diseño de la arquitectura.....	5
Diagrama de la arquitectura del POC.....	6
4.1 Justificación: orquestación sobre coreografía.....	6
4.2 Mecanismo de resiliencia.....	7
5. Implementación.....	7
6. Pruebas y resultados.....	7
6.1 Escenario 4 en detalle: ciclo de vida del Circuit Breaker.....	8
7. Análisis crítico.....	9
7.1 Orquestación vs. coreografía.....	9
7.2 Circuit Breaker: hallazgo sobre fallos de negocio vs. fallos de infraestructura.....	9
7.3 Ventajas y desventajas generales.....	9
Saga (orquestación).....	9
Circuit Breaker.....	9
8. Conclusiones.....	11
Referencias.....	12
Anexos (repositorio, video y evidencias de la practica).....	13
1. Repositorio de github:.....	13
2. Link del video:.....	13
3. Capturas de pantallas de la implementación:.....	13
3.1. Escenario 1: flujo feliz.....	13
3.2. Escenario 2: Fallo en servicio de pago (compensación corta).....	14
3.3. Escenario 3: Fallo en el servicio de inventario (compensación completada).....	15
3.4. Escenario 4: Circuit Breaker abriéndose.....	17

1. Introducción

En las arquitecturas de microservicios, cada servicio gestiona su propia base de datos de forma autónoma, lo que elimina la posibilidad de usar transacciones ACID distribuidas tradicionales entre servicios. A esto se suma el riesgo de fallos en cascada cuando un servicio dependiente deja de responder, lo cual puede comprometer la disponibilidad de todo el sistema. Este informe documenta el diseño, la implementación y las pruebas de una prueba de concepto (PoC) que aborda ambos problemas mediante dos patrones complementarios: el patrón Saga, para mantener la consistencia eventual de los datos a través de transacciones locales encadenadas, y el patrón Circuit Breaker, para prevenir fallos en cascada y mejorar la resiliencia del sistema ante dependencias que fallan.

El caso de estudio es un flujo de compra en un comercio electrónico compuesto por cuatro microservicios de negocio (Order, Payment, Inventory y Shipping) coordinados por un quinto servicio, el orquestador de la Saga.

2. Objetivos

- Diseñar una arquitectura de microservicios capaz de sostener transacciones distribuidas.
- Implementar una Saga mediante orquestación, incluyendo sus transacciones compensatorias.
- Implementar un Circuit Breaker con sus tres estados (Closed, Open, Half-Open) sobre las llamadas entre servicios.
- Simular fallos de negocio y de infraestructura, y verificar el comportamiento resiliente del sistema.
- Analizar críticamente las ventajas, desventajas y contextos de aplicación de ambos patrones.

3. Marco teórico

3.1 Patrón Saga

Richardson (2018) define una saga como una secuencia de transacciones locales en la que cada transacción actualiza los datos dentro de un único servicio y publica un evento o mensaje que dispara la siguiente transacción local de la saga. Si una transacción local falla, la saga ejecuta una serie de transacciones compensatorias que deshacen el efecto de las transacciones anteriores. Esta definición se recoge también en el catálogo de patrones de microservicios de Richardson, donde se describen explícitamente las dos variantes de coordinación: **coreografía**, en la que los servicios se coordinan mediante la publicación y suscripción de eventos sin un control central, y **orquestación**, en la que un orquestador centraliza la lógica de la transacción y le indica a cada participante qué hacer.

En esta práctica se eligió la variante de orquestación: un servicio dedicado (OrchestratorService) invoca secuencialmente a Order, Payment, Inventory y Shipping, y es responsable de disparar las compensaciones cuando un paso falla. Esta decisión se justifica más adelante en la sección 4.

3.2 Patrón Circuit Breaker

Fowler (2014) describe el Circuit Breaker como un objeto que envuelve una llamada a una función protegida y monitorea sus fallos; una vez que los fallos alcanzan cierto umbral, el circuit breaker se "abre" y todas las llamadas posteriores retornan un error de inmediato, sin siquiera intentar la llamada protegida. El propósito es evitar que un cliente agote recursos (hilos, conexiones) esperando respuestas de un servicio que, con alta probabilidad, va a fallar.

El centro de arquitectura de Azure (Microsoft, s.f.) formaliza el patrón en tres estados que imitan el comportamiento de un interruptor eléctrico: Closed (las llamadas pasan con normalidad mientras se cuentan los fallos), Open (las llamadas se rechazan de inmediato sin invocar al servicio real) y Half-Open (tras un periodo de espera, se permite un número limitado de llamadas de prueba para decidir si el servicio se recuperó). Esta misma máquina de estados es la que implementa la librería **Resilience4j** (s.f.), usada en este proyecto a través del starter de abstracción spring-cloud-starter-circuitbreaker-resilience4j (VMware, s.f.), que permite configurar el comportamiento del breaker de forma declarativa y exponerlo mediante Spring Boot Actuator.

Nygard (2018), en su obra de referencia sobre estabilidad de sistemas en producción, enmarca el Circuit Breaker como uno de varios "patrones de estabilidad" que deben combinarse (junto con Timeout, Retry, Bulkhead y Fallback) para que un sistema distribuido tolere fallos parciales sin que estos se propaguen al resto de la arquitectura.

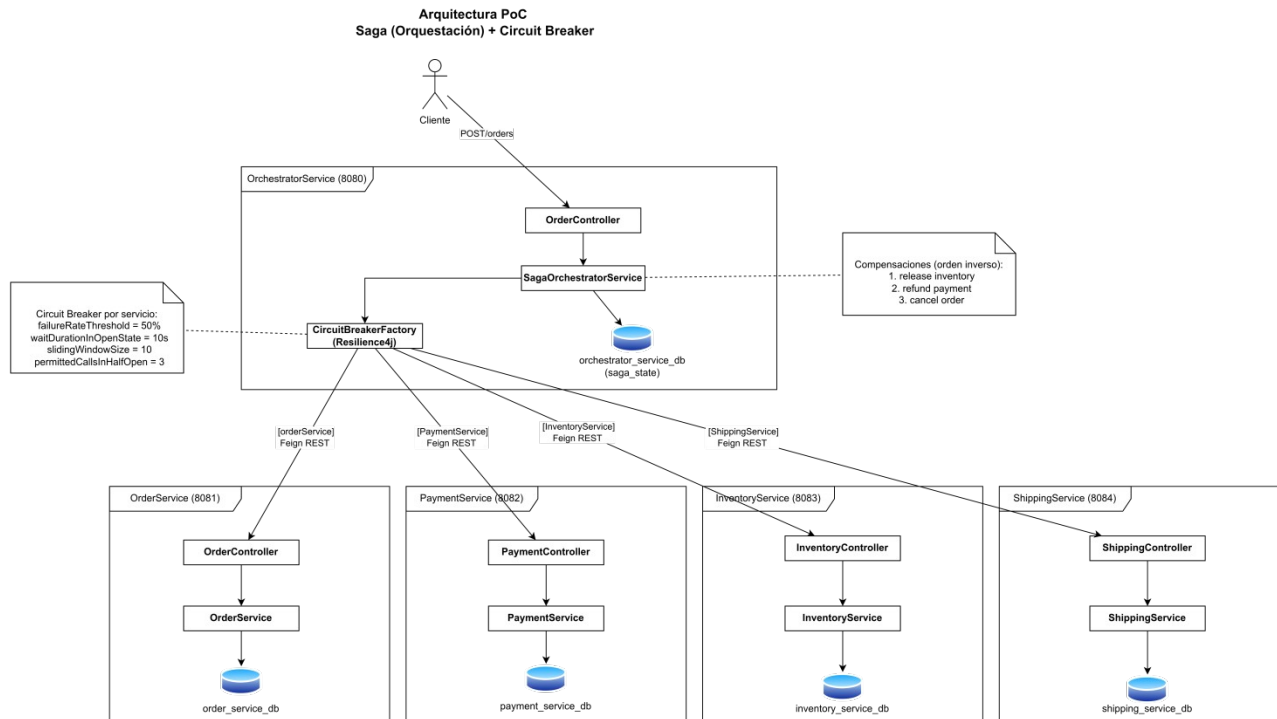
4. Diseño de la arquitectura

El sistema está compuesto por cinco servicios Spring Boot independientes, cada uno con su propia base de datos MySQL:

1. Order Service (puerto 8081): crea y cancela órdenes.

2. Payment Service (puerto 8082): cobra y reembolsa pagos.
3. Inventory Service (puerto 8083): reserva y libera stock.
4. Shipping Service (puerto 8084): programa y cancela envíos.
5. Orchestrator Service (puerto 8080): coordina la Saga y expone el punto de entrada al cliente.

Diagrama de la arquitectura del POC



4.1 Justificación: orquestación sobre coreografía

Se optó por orquestación en lugar de coreografía por tres razones prácticas para el alcance de este PoC. Primero, con solo cuatro pasos de negocio, el flujo completo es más fácil de leer y depurar cuando existe un único punto que lo controla explícitamente, en contraste con una coreografía donde la lógica queda distribuida entre los cuatro servicios y es más difícil de rastrear. Segundo, la orquestación no requiere introducir un broker de mensajería (Kafka o RabbitMQ) solo para esta prueba de concepto; las llamadas síncronas vía REST (Feign) son suficientes. Tercero, al centralizar el estado de la saga en una tabla (saga_state) dentro del propio orquestador, es posible mostrar con claridad, en una demostración en video, el camino feliz y cada escenario de compensación.

Sin embargo, esto hace que el orquestador se convierte en un punto de acoplamiento y, potencialmente, en un cuello de botella si la lógica de negocio crece; en un sistema de producción con más pasos o mayor necesidad de desacoplamiento, la coreografía basada en eventos sería la alternativa a evaluar.

4.2 Mecanismo de resiliencia

Cada llamada saliente del orquestador hacia un servicio de negocio está envuelta en un Circuit Breaker independiente (orderService, paymentService, inventoryService, shippingService), todos creados a partir de una configuración por defecto común. La tabla 1 resume los parámetros aplicados.

Parámetro	Valor configurado	Breve descripción
failureRateThreshold	50%	% de fallos para abrir
waitDurationInOpenState	10 segundos	tiempo antes de pasar a Half-Open
slidingWindowSize	10 llamadas	número de llamadas de prueba
permittedNumberOfCallsInHalfOpenState	3 llamadas	llamadas de prueba

Tabla 1. Parámetros de configuración del Circuit Breaker.

5. Implementación

El stack tecnológico utilizado fue Java 21, Spring Boot 4.1.1, Spring Data JPA, MySQL (una base de datos por servicio) y Lombok. Para la comunicación entre el orquestador y los servicios de negocio se usaron Feign Clients (spring-cloud-starter-openfeign), y para el Circuit Breaker, la abstracción de Spring Cloud sobre Resilience4j (spring-cloud-starter-circuitbreaker-resilience4j).

La Saga se implementó de forma programática: el método createOrderSaga() del orquestador invoca cada paso envuelto en su Circuit Breaker correspondiente mediante CircuitBreakerFactory.create(nombre_del_servicio).run(...). Si un paso falla, ya sea por una excepción de negocio (por ejemplo, pago rechazado o falta de stock) o porque el Circuit Breaker rechaza la llamada por estar abierto (se dispara la cadena de compensación correspondiente en orden inverso): liberar inventario (si se había reservado), reembolsar el pago (si se había cobrado) y cancelar la orden. El estado de cada saga se persiste en la tabla saga_state con los valores STARTED, ORDER_CREATED, PAYMENT_COMPLETED, INVENTORY_RESERVED, SHIPPING_SCHEDULED, COMPLETED, COMPENSATING y FAILED.

Durante la implementación se presentaron dos hallazgos relevantes de compatibilidad de versiones, documentados aquí porque son evidencia del proceso de desarrollo:

- Spring Boot 4.1.1 reemplazó spring-boot-starter-web por spring-boot-starter-webmvc como parte de su modularización.
- Spring Boot 4.0 renombró spring-boot-starter-aop a spring-boot-starter-aspectj; en esta práctica no fue necesario incluir ninguno de los dos, ya que el Circuit Breaker se invoca de forma programática (CircuitBreakerFactory) y no mediante anotaciones estilo AOP (@CircuitBreaker), por lo que no se requiere soporte de proxies AOP/AspectJ.

6. Pruebas y resultados

Se ejecutaron cuatro escenarios de prueba, resumidos en la tabla 2.

Escenario	Descripción	Resultado esperado	Resultado
-----------	-------------	--------------------	-----------

			obtenido
1	Flujo feliz (PROD-001, pago válido)	Orden CONFIRMED, saga COMPLETED	Conforme
2	Fallo simulado en Payment (simulatePaymentFailure=true)	Compensación corta: cancelar orden	Conforme
3	Fallo real en Inventory (PROD-002 sin stock)	Compensación completa: reembolso + cancelar orden	Conforme
4	Payment Service caído, ráfaga de solicitudes	CLOSED -> OPEN -> HALF_OPEN -> CLOSED	Conforme

Tabla 2. Resumen de los escenarios de prueba ejecutados.

6.1 Escenario 4 en detalle: ciclo de vida del Circuit Breaker

Para este escenario se detuvo manualmente el Payment Service y se enviaron más de diez solicitudes consecutivas al orquestador. La tabla 3 reconstruye la secuencia de transiciones de estado observada en el endpoint /actuador/circuitbreakerevents/paymentService.

Hora (aprox.)	Transición	Disparador
23:26:12	CLOSED -> OPEN	FAILURE_RATE_EXCEEDED tras superar 50% de fallos en la ventana de 10 llamadas
23:26:37	OPEN -> HALF_OPEN	Vencimiento de waitDurationInOpenState (10s)
23:26:38	HALF_OPEN -> OPEN	Las 3 llamadas de prueba fallaron (Payment seguía caído)
23:29:51	OPEN -> HALF_OPEN	Nuevo vencimiento del timeout
23:30:16	HALF_OPEN -> CLOSED	Las 3 llamadas de prueba fueron exitosas (Payment reactivado)
23:33:01	CLOSED -> OPEN	Ciclo reproducido: Payment apagado nuevamente
23:38:00	HALF_OPEN -> CLOSED	Ciclo completo confirmado por segunda vez

Tabla 3. Transiciones de estado observadas en el circuit breaker paymentService.

La secuencia observada corresponde exactamente a la especificación teórica del patrón: al superarse el umbral de fallos (failureRateThreshold = 50%) dentro de la ventana deslizante de 10 llamadas, el breaker pasa de Closed a Open y las llamadas posteriores se rechazan de inmediato (eventos NOT_PERMITTED), sin intentar contactar a Payment Service. Tras el periodo de espera configurado (10 segundos), el breaker pasa a Half-Open y permite exactamente tres llamadas de prueba; si fallan, vuelve a Open (como ocurrió en la primera iteración, mientras Payment seguía apagado), y si tienen éxito, cierra el circuito (como ocurrió tras reactivar el servicio). El ciclo se reprodujo dos veces de forma consistente.

7. Análisis crítico

7.1 Orquestación vs. coreografía

La orquestación demostró ser adecuada para este caso de cuatro pasos: permitió centralizar el estado de la saga en una sola tabla y facilitó enormemente la depuración durante las pruebas, al poder consultar `GET /orders/{id}/status` y ver de inmediato en qué paso falló una saga. El costo observado fue el acoplamiento del orquestador a los contratos (DTOs) de los cuatro servicios; cualquier cambio en el contrato de un servicio de negocio obliga a actualizar también los DTOs y Feign Clients del orquestador, duplicación que en una coreografía basada en eventos no existiría de la misma forma.

7.2 Circuit Breaker: hallazgo sobre fallos de negocio vs. fallos de infraestructura

Durante las pruebas se identificó una limitación de diseño en la implementación actual: el Circuit Breaker del Payment Service contabiliza como "fallo" tanto los rechazos de pago simulados (`simulatePaymentFailure = true`, una respuesta de negocio legítima en la que el servicio sí respondió) como las caídas reales de infraestructura (servicio apagado, conexión rehusada). Esto es visible en el registro de eventos: antes de los eventos de "Conexión rehusada" que finalmente abrieron el circuito, ya se habían contabilizado varios eventos ERROR por pagos rechazados intencionalmente (HTTP 422).

Desde el punto de vista de diseño, esto es una debilidad real: un Circuit Breaker debería protegerse de fallos de infraestructura (el servicio no responde o responde con error del lado del servidor), no de respuestas de negocio válidas (el servicio respondió, simplemente la operación fue rechazada). Si, por ejemplo, una proporción alta de clientes tuviera fondos insuficientes en un entorno real, el circuito podría abrirse innecesariamente y bloquear pagos que sí podrían procesarse con éxito para otros clientes. Resilience4j permite corregir este comportamiento excluyendo del conteo ciertas excepciones de negocio mediante la opción `ignoreExceptions()` del `CircuitBreakerConfig`; esta mejora se identifica como trabajo futuro del PoC.

7.3 Ventajas y desventajas generales

Saga (orquestación)

- Ventajas: flujo centralizado y fácil de depurar; trazabilidad clara del estado de cada transacción distribuida; no requiere infraestructura de mensajería adicional.
- Desventajas: el orquestador concentra lógica de negocio y acoplamiento; puede convertirse en un cuello de botella o en un "mini monolito" si no se diseña con cuidado; es un punto adicional de fallo.

Circuit Breaker

- Ventajas: evita el agotamiento de recursos ante dependencias caídas; falla rápido en vez de esperar timeouts largos; permite recuperación automática y gradual mediante el estado Half-Open; es observable (Actuador) en tiempo real.
- Desventajas: requiere calibrar correctamente sus parámetros según el comportamiento esperado del servicio protegido; si no se distingue entre tipos de error, puede abrirse por motivos que no reflejan una falla real de infraestructura, como se evidenció en la sección 7.2.

8. Conclusiones

El PoC cumplió los objetivos planteados: se implementó una Saga orquestada capaz de ejecutar compensaciones completas ante fallos en cualquiera de sus pasos, y se configuraron cuatro Circuit Breakers independientes que exhibieron el ciclo completo Closed -> Open -> Half-Open -> Closed de forma reproducible y verificable mediante Spring Boot Actuator.

El ejercicio también permitió identificar, de forma empírica y no solo teórica, una limitación real de la implementación (la falta de distinción entre fallos de negocio y de infraestructura en el conteo del Circuit Breaker) que ilustra por qué ambos patrones exigen un diseño cuidadoso y no una aplicación mecánica de la configuración por defecto.

Referencias

Fowler, M. (2014, 6 de marzo). *CircuitBreaker*. <https://martinfowler.com/bliki/CircuitBreaker.html>

Microsoft. (s.f.). *Circuit Breaker pattern*. Azure Architecture Center. Microsoft Learn.
<https://learn.microsoft.com/azure/architecture/patterns/circuit-breaker>

Nygard, M. T. (2018). *Release it!: Design and deploy production-ready software* (2.^a ed.). Pragmatic Bookshelf.

[https://repo.darmajaya.ac.id/4586/1/
Release%20It%21%20Design%20and%20Deploy%20Production-
Ready%20Software%20%28%20PDFDrive%20%29.pdf](https://repo.darmajaya.ac.id/4586/1/Release%20It%21%20Design%20and%20Deploy%20Production-Ready%20Software%20%28%20PDFDrive%20%29.pdf)

Resilience4j. (s.f.). *CircuitBreaker*. Resilience4j User Guide.
<https://resilience4j.readme.io/docs/circuitbreaker>

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

[https://github.com/AAAAAStudy/bookshelf-1/blob/main/Extra/
Microservices%20Patterns%20With%20examples%20in%20Java.pdf](https://github.com/AAAAAStudy/bookshelf-1/blob/main/Extra/Microservices%20Patterns%20With%20examples%20in%20Java.pdf)

Richardson, C. (s.f.). *Pattern: Saga*. Microservices.io.
<https://microservices.io/patterns/data/saga>

VMware. (s.f.). *Spring Cloud Circuit Breaker reference documentation*. Spring.io.
<https://docs.spring.io/spring-cloud-circuitbreaker/docs/current/reference/html/>

Anexos (repositorio, video y evidencias de la practica)

1. Repositorio de github:

[git@github.com:luisbaquiaux/practica5_sa.git](https://github.com/luisbaquiaux/practica5_sa.git)

2. Link del vide:

https://drive.google.com/drive/folders/1YfO0XJy3EnLyrvWRgCuurzlfBWTqmdVX?usp=drive_link

3. Presentación:

https://luisbaquiaux1995.github.io/saga_circuit_breaker/

4. Capturas de pantallas de la implementación:

- 4.1. Escenario 1: flujo feliz

Se hace la petición, y de momento no hay ninguna advertencia o fallo.

The screenshot shows a REST client interface with a dark theme. At the top, there are tabs for different endpoints: "POST create order", "GET orders status", "GET actuador", and "GET transciones de esta...". The active tab is "POST create order". Below the tabs, the request details are shown: "POST http://localhost:8080/orders". The request body is a JSON object with the following fields: "customerId": 1001, "productId": "PROD-001", "quantity": 1, "totalAmount": 149.99, "address": "Quetzaltenango", and "simulatePaymentFailure": false. The response is also shown as a JSON object with the following fields: "orderId": 48, "status": "COMPLETED", and "failureReason": null.

```
POST http://localhost:8080/orders

{
  "customerId": 1001,
  "productId": "PROD-001",
  "quantity": 1,
  "totalAmount": 149.99,
  "address": "Quetzaltenango",
  "simulatePaymentFailure": false
}
```

```
{
  "orderId": 48,
  "status": "COMPLETED",
  "failureReason": null
}
```

El estado de la saga este escenario es completada.

The screenshot shows a database viewer interface with a dark theme. At the top, there are tabs for different databases: "shippings [mysql]" and "saga_state [mysql]". The active tab is "saga_state [mysql]". Below the tabs, the table data is shown in a grid format. The columns are "order_id", "status", and "updated_at". The data row shows "48", "COMPLETED", and "2026-10-03 01:11:13.945368".

order_id	status	updated_at
48	COMPLETED	2026-10-03 01:11:13.945368

Datos de la orden confirmada

```
Database changed
mysql> select * from orders
-> ;
+-----+-----+-----+-----+-----+-----+
| id | created_at          | customer_id | product_id | quantity | status   | total_amount |
+-----+-----+-----+-----+-----+-----+
| 48 | 2026-10-03 01:11:13.057874 |      1001 | PROD-001   |         1 | CONFIRMED |        149.99 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Pago completado

```
Database changed
mysql> select * from payments;
+-----+-----+-----+-----+-----+-----+
| id | amount | created_at          | customer_id | order_id | status   |
+-----+-----+-----+-----+-----+-----+
| 13 | 149.99 | 2026-10-03 01:11:13.364631 |      1001 |        48 | COMPLETED |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

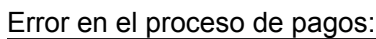
Envío programado

```
Database changed
mysql> select * from shippings;
+-----+-----+-----+-----+-----+-----+
| id | address          | created_at          | order_id | status   |
+-----+-----+-----+-----+-----+-----+
| 9  | Quetzaltenango  | 2026-10-03 01:11:13.829790 |        48 | SCHEDULED |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

4.2. Escenario 2: Fallo en servicio de pago (compensación corta)

En este escenario se realiza un fallo a propósito para ver los diferentes estados en los servicios.

Realización de la petición:



El resto de servicios no se actualiza ningún porque a partir de payment ha fallado y la orden se cancela.

```
Database changed
mysql> select * from orders;
+-----+-----+-----+-----+-----+-----+-----+
| id | created_at | customer_id | product_id | quantity | status | total_amount |
+-----+-----+-----+-----+-----+-----+-----+
| 48 | 2026-10-03 01:11:13.057874 | 1001 | PROD-001 | 1 | CONFIRMED | 149.99 |
| 49 | 2026-10-03 01:22:12.890532 | 1001 | PROD-001 | 1 | CANCELLED | 149.99 |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

4.3. Escenario 3: Fallo en el servicio de inventario (compensación completada)

Envío de la petición:

```
POST http://localhost:8080/orders

Body *
{
  "customerId": 1001,
  "productId": "PROD-002",
  "quantity": 1,
  "totalAmount": 149.99,
  "address": "Quetzaltenango",
  "simulatePaymentFailure": false
}
```

Fallo en inventario por falta de stok:

```
Response >> {} JSON 422 Unprocessable Entity 395ms 362B

1 {
2   "error": "Saga fallida en Inventory: Inventory Service falló: [409] d
   uring [POST] to [http://localhost:8083/inventory/reserve] [InventoryCli
   ent#reserve(InventoryRequestDTO)]: [{\"error\": \"No hay suficiente stoc
   k para la orden PROD-002\", \"timestamp\": \"2026-10-03T01:33:15.36212525
   3\", \"status\": 409}]",
3   "timestamp": "2026-10-03T01:33:15.703373272",
4   "status": 422
5 }
```

Se crea la orden, se cobra, el servicio de inventario lanza la excepción por falta de stock, se realiza el reembolso y se cancela la orden.

```
Database changed
mysql> select * from stock;
+-----+-----+
| product_id | available_quantity |
+-----+-----+
| PROD-001   | 6                  |
| PROD-002   | 0                  |
+-----+-----+
2 rows in set (0.00 sec)
```

No existe stock para el producto 2, y por eso se debería lanzar la excepción.

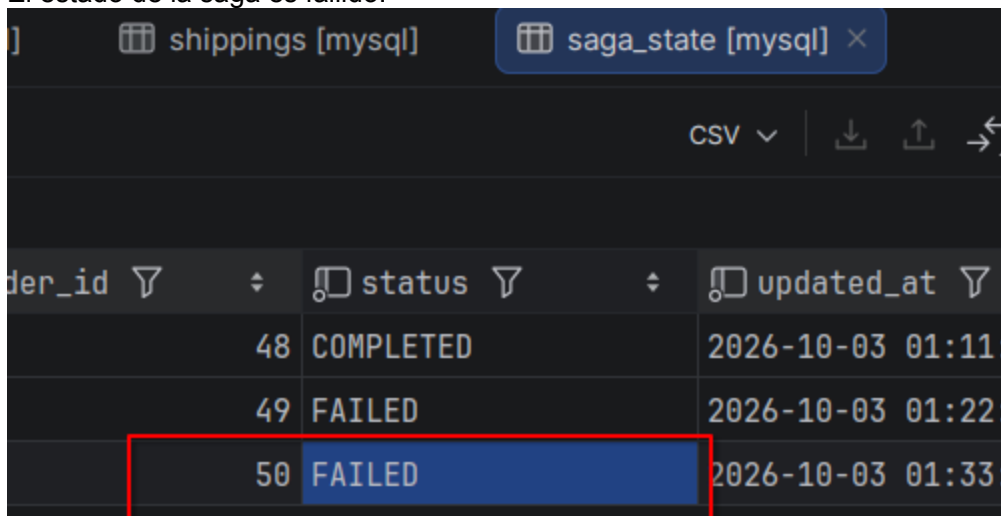
```
Database changed
mysql> select * from payments;
+-----+-----+-----+-----+-----+-----+
| id | amount | created_at | customer_id | order_id | status |
+-----+-----+-----+-----+-----+-----+
| 13 | 149.99 | 2026-10-03 01:11:13.364631 | 1001 | 48 | COMPLETED |
| 14 | 149.99 | 2026-10-03 01:33:15.337122 | 1001 | 50 | REFUNDED |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

El pago que se había realizado se reembolsa.

```
Database changed
mysql> select * from orders;
+-----+-----+-----+-----+-----+-----+-----+
| id | created_at | customer_id | product_id | quantity | status | total_amount |
+-----+-----+-----+-----+-----+-----+-----+
| 48 | 2026-10-03 01:11:13.057874 | 1001 | PROD-001 | 1 | CONFIRMED | 149.99 |
| 49 | 2026-10-03 01:22:12.890532 | 1001 | PROD-001 | 1 | CANCELLED | 149.99 |
| 50 | 2026-10-03 01:33:15.316787 | 1001 | PROD-002 | 1 | CANCELLED | 149.99 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

La orden se canceló.

El estado de la saga es fallido:



order_id	status	updated_at
48	COMPLETED	2026-10-03 01:11:13.364631
49	FAILED	2026-10-03 01:22:12.890532
50	FAILED	2026-10-03 01:33:15.337122

4.4. Escenario 4: Circuit Breaker abriéndose

1. Se apaga el servicio de pagos.

```
~/Documentos/courses/software_avanzado/practica5/PaymentService (35m 2.24s)
./mvnw spring-boot:run

Started PaymentServiceApplication in 4.975 seconds (process running for 5.448)
2026-10-03T01:11:13.241-06:00 INFO 39016 --- [payment-service] [nio-8082-exec-1] o.a.c.c.C
  Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-03T01:11:13.242-06:00 INFO 39016 --- [payment-service] [nio-8082-exec-1] o.s.web.s
  Initializing Servlet 'dispatcherServlet'
2026-10-03T01:11:13.243-06:00 INFO 39016 --- [payment-service] [nio-8082-exec-1] o.s.web.s
  Completed initialization in 1 ms
Hibernate: insert into payments (amount,created_at,customer_id,order_id,status) values (?,?
2026-10-03T01:22:12.916-06:00 WARN 39016 --- [payment-service] [nio-8082-exec-3] .m.m.a.Ex
  Resolved [org.luisbaquiaux.payment.service.exception.PaymentProcessingException: Error al pr
Hibernate: insert into payments (amount,created_at,customer_id,order_id,status) values (?,?
Hibernate: select p1_0.id,p1_0.amount,p1_0.created_at,p1_0.customer_id,p1_0.order_id,p1_0.s
_0.order_id=?
Hibernate: update payments set amount=?,customer_id=?,order_id=?,status=? where id=?
^C2026-10-03T01:43:08.607-06:00 INFO 39016 --- [payment-service] [ionShutdownHook] o.s.bo
  : Commencing graceful shutdown. Waiting for active requests to complete
2026-10-03T01:43:08.612-06:00 INFO 39016 --- [payment-service] [tomcat-shutdown] o.s.boot
  Graceful shutdown complete
2026-10-03T01:43:08.625-06:00 INFO 39016 --- [payment-service] [ionShutdownHook] j.LocalCo
  Closing JPA EntityManagerFactory for persistence unit 'default'
2026-10-03T01:43:08.635-06:00 INFO 39016 --- [payment-service] [ionShutdownHook] com.zaxxe
  HikariPool-1 - Shutdown initiated...
2026-10-03T01:43:08.644-06:00 INFO 39016 --- [payment-service] [ionShutdownHook] com.zaxxe
  HikariPool-1 - Shutdown completed.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 35:00 min
[INFO] Finished at: 2026-10-03T01:43:08-06:00
[INFO] -----

[main] 0
```

2. Se realiza las 10 peticiones

```
practica 5
POST create or GET orders status GET actuator GET transiciones de esta... +
POST http://localhost:8080/orders
Send

Params Body Headers Auth Vars Script Assert Tests Docs File Settings >> JSON Prettify
1 {
2   "customerId": 1001,
3   "productId": "PROD-001",
4   "quantity": 1,
5   "totalAmount": 149.99,
6   "address": "Quetzaltenango",
7   "simulatePaymentFailure": false
8 }

Response >> JSON 422 Unprocessable Entity 54ms 182B
1 {
2   "error": "Saga fallida en Payment: Payment Service falló: Conexión re
3   husada executing POST http://localhost:8082/payments",
4   "timestamp": "2026-10-03T01:44:58.620295022",
5   "status": 422
}
```

3. El servicio de pagos no responde por tanto las ordenes fueron canceladas, además el orquestador ejecuta las operaciones de compensación de cada servicio.

```
mysql> select * from orders;
```

id	created_at	customer_id	product_id	quantity	status	total_amount
48	2026-10-03 01:11:13.057874	1001	PROD-001	1	CONFIRMED	149.99
49	2026-10-03 01:22:12.890532	1001	PROD-001	1	CANCELLED	149.99
50	2026-10-03 01:33:15.316787	1001	PROD-002	1	CANCELLED	149.99
51	2026-10-03 01:44:58.575948	1001	PROD-001	1	CANCELLED	149.99
52	2026-10-03 01:46:47.062192	1001	PROD-001	1	CANCELLED	149.99
53	2026-10-03 01:46:47.938473	1001	PROD-001	1	CANCELLED	149.99
54	2026-10-03 01:46:48.737866	1001	PROD-001	1	CANCELLED	149.99
55	2026-10-03 01:46:49.434344	1001	PROD-001	1	CANCELLED	149.99
56	2026-10-03 01:46:50.150255	1001	PROD-001	1	CANCELLED	149.99
57	2026-10-03 01:46:50.858202	1001	PROD-001	1	CANCELLED	149.99
58	2026-10-03 01:46:51.517134	1001	PROD-001	1	CANCELLED	149.99
59	2026-10-03 01:46:52.148277	1001	PROD-001	1	CANCELLED	149.99
60	2026-10-03 01:46:52.787756	1001	PROD-001	1	CANCELLED	149.99

4. El estado de payment-service ha pasado de 'closed' a 'open', esto recuperado en (por el actuador): <http://localhost:8080/actuador/circuitbreakers> o en <http://localhost:8080/actuador/circuitbreakerevents/paymentService>

```

20     "notPermittedCalls": 0,
21     "slowCallRate": "0.0%",
22     "slowCallRateThreshold": "100.0%",
23     "slowCalls": 0,
24     "slowFailedCalls": 0,
25     "state": "CLOSED"
26 },
27 "paymentService": {
28     "bufferedCalls": 10,
29     "failedCalls": 8,
30     "failureRate": "80.0%",
31     "failureRateThreshold": "50.0%",
32     "notPermittedCalls": 3,
33     "slowCallRate": "0.0%",
34     "slowCallRateThreshold": "100.0%",
35     "slowCalls": 0,
36     "slowFailedCalls": 0,
37     "state": "OPEN"
38 },
39 "shippingService": {

```



```

19     "failureRateThreshold": "50.0%",
20     "notPermittedCalls": 0,
21     "slowCallRate": "0.0%",
22     "slowCallRateThreshold": "100.0%",
23     "slowCalls": 0,
24     "slowFailedCalls": 0,
25     "state": "CLOSED"
26 },
27 "paymentService": {
28     "bufferedCalls": 1,
29     "failedCalls": 0,
30     "failureRate": "-1.0%",
31     "failureRateThreshold": "50.0%",
32     "notPermittedCalls": 0,
33     "slowCallRate": "-1.0%",
34     "slowCallRateThreshold": "100.0%",
35     "slowCalls": 0,
36     "slowFailedCalls": 0,
37     "state": "HALF_OPEN"
38 },
39 "shippingService": {
40     "bufferedCalls": 6,
41     "failedCalls": 0,
42     "failureRate": "-1.0%",
43     "failureRateThreshold": "50.0%",
44     "notPermittedCalls": 0,
45     "slowCallRate": "-1.0%",
46     "slowCallRateThreshold": "100.0%",

```

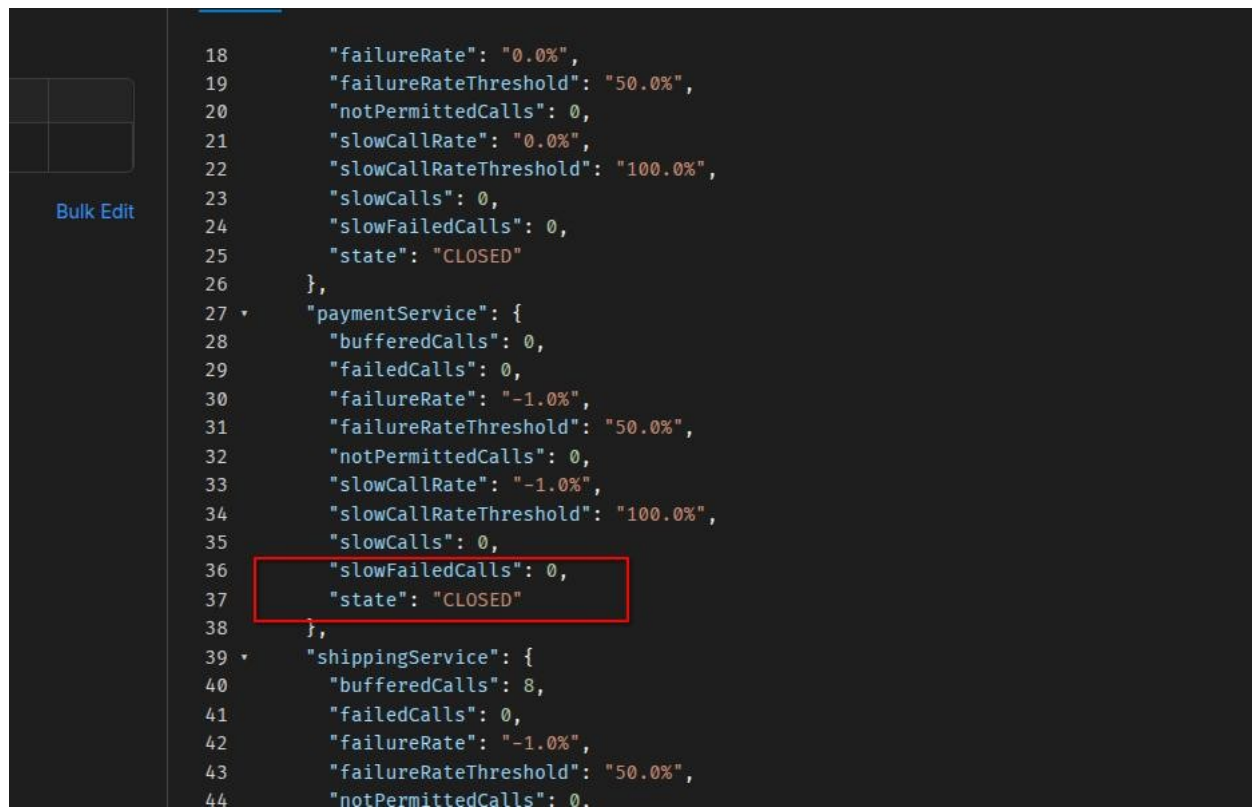
7. Se realiza las 3 peticiones de prueba para saber si está disponible el servicio y el estado de payment-service pasa de 'half_open' a 'closed'

```

177     "type": "SUCCESS"
178 },
179 {
180     "circuitBreakerName": "paymentService",
181     "creationTime": "2026-10-03T01:58:59.600268552-06:00[America/Guatemala]",
182     "durationInMs": 16,
183     "errorMessage": null,
184     "stateTransition": null,
185     "type": "SUCCESS"
186 },
187 {
188     "circuitBreakerName": "paymentService",
189     "creationTime": "2026-10-03T01:59:00.496528244-06:00[America/Guatemala]",
190     "durationInMs": 13,
191     "errorMessage": null,
192     "stateTransition": null,
193     "type": "SUCCESS"
194 },
195 {
196     "circuitBreakerName": "paymentService",
197     "creationTime": "2026-10-03T01:59:00.496864794-06:00[America/Guatemala]",
198     "durationInMs": null,
199     "errorMessage": null,
200     "stateTransition": "State transition from HALF_OPEN to CLOSED",
201     "type": "STATE_TRANSITION"
202 }
203 ]
204 }

```

8. En el resto de peticiones (según las configuraciones) el estado de payment-service cambia a 'closed':



```
18     "failureRate": "0.0%",
19     "failureRateThreshold": "50.0%",
20     "notPermittedCalls": 0,
21     "slowCallRate": "0.0%",
22     "slowCallRateThreshold": "100.0%",
23     "slowCalls": 0,
24     "slowFailedCalls": 0,
25     "state": "CLOSED"
26 },
27   "paymentService": {
28     "bufferedCalls": 0,
29     "failedCalls": 0,
30     "failureRate": "-1.0%",
31     "failureRateThreshold": "50.0%",
32     "notPermittedCalls": 0,
33     "slowCallRate": "-1.0%",
34     "slowCallRateThreshold": "100.0%",
35     "slowCalls": 0,
36     "slowFailedCalls": 0,
37     "state": "CLOSED"
38   },
39   "shippingService": {
40     "bufferedCalls": 8,
41     "failedCalls": 0,
42     "failureRate": "-1.0%",
43     "failureRateThreshold": "50.0%",
44     "notPermittedCalls": 0,
```

```
322     },
323     {
324         "circuitBreakerName": "paymentService",
325         "creationTime": "2026-10-03T02:06:11.663382777-06:00[America/Guatemala]",
326         "durationInMs": 12,
327         "errorMessage": null,
328         "stateTransition": null,
329         "type": "SUCCESS"
330     },
331     {
332         "circuitBreakerName": "paymentService",
333         "creationTime": "2026-10-03T02:06:12.277191258-06:00[America/Guatemala]",
334         "durationInMs": 12,
335         "errorMessage": null,
336         "stateTransition": null,
337         "type": "SUCCESS"
338     },
339     {
340         "circuitBreakerName": "paymentService",
341         "creationTime": "2026-10-03T02:06:12.277245042-06:00[America/Guatemala]",
342         "durationInMs": null,
343         "errorMessage": null,
344         "stateTransition": "State transition from HALF_OPEN to CLOSED",
345         "type": "STATE_TRANSITION"
346     }
347 ]
348 }
```